BookFlow AI Commerce

Plataforma Inteligente de Catalogación, Pricing Competitivo y Venta de Libros

Documento maestro del proyecto

Enfoque empresarial para ejecución en células de desarrollo

Dominio: comercio de libros y publicaciones seriadas

Tecnologías base: Python, FastAPI, React, PostgreSQL, Docker

Tipo de documento	Propuesta integral de producto digital
Perfil de equipo	Células Scrum de 7 a 9 developers
Duración	3 sprints de 2 semanas
Nivel de reto	Intermedio–intermedio
Propósito	Implementación, evaluación por sprint

Versión formal consolidada

Secciones del Documento	Qué explica
Arquitectura	cómo está construido
Vista lógica	cómo se divide el sistema
Flujos	qué pasa paso a paso
Módulos	qué capacidades existen
Rutas	cómo se usa el sistema
Principios Operativos	De cumplimiento por los estudiantes
Backlog Product	Actividades configuradas por Sprint

1. Resumen ejecutivo

Propósito: Definir con claridad qué se construye, cómo se construye, por qué se construye y cuál es el papel real de cada componente dentro de la solución.

BookFlow AI Commerce es una plataforma de software orientada a transformar inventarios bibliográficos dispersos, incompletos o de baja calidad en un ecosistema digital comercializable, trazable y operable. El sistema no se limita a "mostrar libros en una web"; su propósito es construir una cadena completa de valor que va desde la carga y normalización del inventario hasta la generación de pedidos y la atención asistida al cliente.

La solución se diseña para organizaciones que manejan libros, publicaciones seriadas, saldos editoriales o material usado, y que enfrentan dificultades para organizar su inventario, enriquecer metadatos, definir precios competitivos y operar un flujo comercial de manera digital y consistente. En este contexto, BookFlow AI Commerce busca resolver el problema de fondo: convertir datos desordenados en catálogo útil, y catálogo útil en operación comercial.

La plataforma integra cuatro grandes capacidades:

Primera capacidad: gestión estructurada de inventario.
Permite importar archivos de inventario (XLS, CVS), validar su estructura, detectar errores, registrar trazabilidad por lote y construir una base confiable de productos. Esto evita que el sistema dependa de hojas de cálculo ambiguas o de carga manual no controlada.

Segunda capacidad: inteligencia artificial aplicada al dato y al negocio.
La IA no se plantea como un accesorio o un chatbot superficial. Tiene un rol estructural en tres frentes:

- **Enriquecimiento de datos**, completando metadatos bibliográficos a partir de fuentes externas;
- **Pricing inteligente**, apoyando la definición de precios sugeridos competitivos con lógica explicable;
- **Asistencia comercial**, respondiendo consultas del usuario sobre disponibilidad, precios, características y alternativas.

Tercera capacidad: exposición comercial del catálogo.
La información procesada no queda en una base de datos aislada. El sistema la convierte en catálogo navegable, filtrable y orientado a la venta, permitiendo que el usuario consulte productos con información suficientemente clara para tomar decisiones.(Resumen del libro, contenidos, autor, edición, año, editorial, stock, estado)

Cuarta capacidad: operación comercial integrada.
El sistema habilita carrito, creación de pedido, validación de disponibilidad y trazabilidad del proceso comercial, con una lógica consistente entre catálogo, inventario y pedido.

Desde el punto de vista técnico, la solución se construye con una **arquitectura de microservicios independientes**, usando:

- **React** para frontend;
- **Python con FastAPI** para backend;
- **PostgreSQL** como motor de persistencia, con aislamiento de base de datos por servicio;
- **Docker Compose** para despliegue e integración local;
- **BFF / API Gateway** como punto único de entrada para el frontend.

El proyecto se ejecuta con una célula de **7 a 9 desarrolladores**, en **3 sprints de 2 semanas**, con enfoque incremental. Esto significa que la solución no se construye como un bloque único, sino como una suma de piezas funcionales y demostrables que evolucionan sprint a sprint.

Este documento no debe entenderse como una descripción ejecutiva ligera. Debe servir como guía de implementación, alineación técnica y entendimiento del sistema para el equipo que lo va a construir.

2. Problema de negocio

Propósito: Describir el problema real que da origen al proyecto, no solo desde lo tecnológico, sino desde la operación del negocio y sus consecuencias.

Las organizaciones que comercializan libros, publicaciones seriadas, material usado o saldos editoriales suelen operar con inventarios mantenidos en hojas de cálculo, archivos locales o registros parciales que no fueron diseñados para sostener un proceso comercial digital. En la práctica, esto significa que los datos del inventario suelen presentar uno o varios de los siguientes problemas:

- títulos duplicados;
- metadatos incompletos;
- inconsistencias entre autor, editorial, año y categoría;
- ausencia de identificadores como ISBN o ISSN (identificador para publicaciones seriadas);
- falta de claridad sobre condición física del ejemplar;
- carencia de una estructura estandarizada para describir defectos;
- precios definidos manualmente sin una lógica trazable (precios de editorial que se deben mantener, precios de libros descatalogados o ediciones antiguas que se deben buscar en otros distribuidores y analizar su precio con IA para tener un precio competitivo).

Esto provoca un problema que va mucho más allá del “desorden de datos”. Cuando el catálogo está mal estructurado, el negocio pierde capacidad de publicación, de búsqueda, de análisis y de respuesta comercial. En otras palabras: no basta con tener inventario; hay que tenerlo listo para vender.

En el caso de material usado, el problema se vuelve más complejo. El valor comercial de un libro no depende únicamente de su título o autor, sino también de su estado físico. Aspectos como desgaste, subrayados, anotaciones, faltantes o deterioro afectan directamente la percepción del cliente y el precio. Si esa información no se registra con claridad, el negocio incurre en reclamos, pérdida de confianza y devoluciones evitables.

Otro problema crítico es el pricing. En muchos casos, el precio se define por intuición, costumbre, copia manual de referencias aisladas o simplemente por redondeo. Esto genera dos riesgos opuestos:

- vender por debajo del valor posible, perdiendo margen;
- publicar por encima del mercado, afectando rotación y conversión.

A esto se suma la fragmentación del flujo comercial. Es común que una organización tenga:

- inventario en un archivo;
- catálogo en otro canal;
- precios definidos aparte;
- atención al cliente por mensajería;
- pedidos registrados manualmente.

El resultado es una operación desarticulada, lenta y propensa al error. El cliente pregunta por un libro, alguien debe buscarlo manualmente, verificar disponibilidad, revisar precio, responder, confirmar pedido y registrar todo por fuera del sistema. Eso no escala y tampoco permite trazabilidad.

Por tanto, el problema de negocio que resuelve BookFlow AI Commerce es el siguiente:

La organización dispone de inventario bibliográfico, pero no cuenta con una plataforma capaz de convertirlo en un catálogo digital enriquecido, valorado inteligentemente y operable comercialmente de forma consistente.

El proyecto busca cerrar esa brecha entre “tener productos” y “tener una operación comercial digital útil”.

3. Justificación empresarial

Propósito: Explicar por qué vale la pena construir la solución y qué beneficios concretos aporta tanto al negocio como al equipo que la desarrolla.

La justificación empresarial del proyecto se apoya en el hecho de que el valor del inventario no se realiza automáticamente por el solo hecho de existir. Un negocio puede tener cientos o miles de libros disponibles y aun así no convertirlos en ventas, si no cuenta con estructura, visibilidad, calidad de datos y proceso comercial. BookFlow AI Commerce se justifica porque ataca precisamente esos cuellos de botella.

3.1 Justificación operativa

La solución reduce fricción operativa al centralizar funciones que normalmente están dispersas:

- carga de inventario;
- limpieza y validación;
- enriquecimiento de información;
- pricing;
- exposición del catálogo;
- creación y seguimiento de pedidos.

Esto disminuye tiempos manuales, reduce reprocesos y mejora la calidad del trabajo comercial. Lo que antes dependía de varias herramientas desconectadas pasa a resolverse desde un flujo integrado.

3.2 Justificación comercial

Desde la perspectiva comercial, la plataforma permite aumentar el valor utilizable del inventario. Un inventario desordenado o mal descrito no rota con facilidad. En cambio, un catálogo enriquecido, con mejores descripciones, precio razonado y disponibilidad visible, tiene mayores probabilidades de conversión.

La presencia de pricing inteligente agrega otra dimensión de valor. El negocio deja de depender únicamente del criterio manual y empieza a apoyarse en referencias, reglas y lógica explicable. Esto no elimina la decisión humana, pero la mejora.

3.3 Justificación tecnológica

Tecnológicamente, el proyecto es valioso porque no se construye como una aplicación cerrada y rígida, sino como una solución modular basada en microservicios independientes. Esto permite:

- separación clara de responsabilidades;
- trabajo paralelo por equipos;
- despliegue desacoplado;
- mantenibilidad;
- posibilidad de evolución futura.

El uso de FastAPI, React y PostgreSQL lo hace además viable para un equipo con experiencia intermedia que ya viene de un reto previo de pedidos y que ahora necesita subir de nivel sin caer en una complejidad imposible.

3.4 Justificación formativa

Este proyecto también tiene un valor formativo muy fuerte. Obliga al equipo a trabajar con:

- arquitectura de microservicios;
- BFF;

- persistencia por servicio;
- integración REST;
- arquitectura hexagonal ligera;
- APIs externas;
- lógica de negocio distribuida;
- componentes de IA aplicada.

Es decir, el proyecto no solo busca producir un resultado funcional, sino elevar el nivel técnico del equipo. En lugar de repetir un CRUD simple, los desarrolladores se enfrentan a problemas más reales: contratos entre servicios, trazabilidad, consistencia entre dominios, enriquecimiento de datos y lógica inteligente aplicable al negocio.

3.5 Justificación estratégica

La plataforma tiene potencial de evolución. La solución podría escalar a:

- librerías;
- distribuidores;
- fondos editoriales;
- catálogos institucionales;
- modelos SaaS a futuro.

4. Objetivo general

Propósito: Establecer el resultado global que el proyecto debe alcanzar.

Diseñar e implementar una plataforma de software empresarial basada en microservicios que permita gestionar inventario bibliográfico, enriquecer metadatos mediante inteligencia artificial, calcular precios sugeridos competitivos, publicar un catálogo comercial navegable y ejecutar un flujo de carrito, pedido y atención asistida al cliente sobre datos reales del sistema.

Este objetivo general integra tanto la dimensión operativa como la dimensión tecnológica. No se trata solo de construir interfaces o exponer APIs, sino de crear una solución coherente que conecte el inventario con la venta y que use IA en funciones concretas de valor.

5. Objetivos específicos

Propósito: Desagregar el objetivo general en capacidades concretas que orienten la implementación.

5.1 Implementar carga y validación de inventario

Desarrollar un mecanismo de carga masiva de inventario a partir de archivos estructurados, capaz de validar formato, detectar errores, registrar trazabilidad por lote y persistir datos válidos en el servicio correspondiente.

5.2 Construir el servicio de catálogo

Desarrollar un microservicio especializado en la gestión de libros y publicaciones, capaz de exponer metadatos comerciales, clasificaciones y fichas navegables para su uso en frontend y en otros servicios del ecosistema.

5.3 Incorporar enriquecimiento bibliográfico con IA

Diseñar e implementar un servicio de enriquecimiento que consuma fuentes externas y mejore la calidad de los registros del catálogo, completando atributos como portada, editorial, resumen, año, categorías o descriptores relevantes.

5.4 Implementar pricing inteligente

Construir un servicio de pricing que combine referencias externas, reglas internas del negocio y condición del producto para generar precios sugeridos trazables y explicables.

5.5 Desarrollar el flujo comercial

Implementar la lógica necesaria para permitir navegación del catálogo, carrito, validación de disponibilidad, creación de pedido y trazabilidad del proceso comercial.

5.6 Incorporar un asistente IA conectado al sistema

Construir un servicio de asistencia capaz de responder preguntas sobre productos, disponibilidad, precio y alternativas usando información real del sistema, no respuestas genéricas desconectadas del negocio.

5.7 Asegurar integración y despliegue reproducible

Integrar los microservicios bajo un BFF y desplegar la solución en contenedores con una configuración controlada y reproducible mediante Docker Compose.

6. Alcance del proyecto

Propósito: Delimitar claramente qué entra en el proyecto y qué no, para evitar expectativas erradas y sobrecarga de implementación.

6.1 Alcance incluido

El proyecto incluye las siguientes capacidades:

a. Gestión de inventario

- carga masiva por archivo;
- validación estructural;
- persistencia de registros;
- trazabilidad por lote;
- detección y visualización de errores.

b. Gestión de catálogo

- CRUD o exposición controlada de productos;
- consulta de libros y publicaciones;
- organización por categorías o clasificaciones;
- construcción de ficha comercial.

c. Enriquecimiento con IA

- consulta a APIs externas;
- homologación y normalización de datos;
- mejora automática del registro bibliográfico;
- actualización del catálogo con información enriquecida.

d. Pricing inteligente

- consulta de referencias externas cuando aplique;
- aplicación de reglas internas;
- ajuste por condición física;
- generación de precio sugerido;
- explicación del cálculo.

e. Catálogo comercial

- navegación web;
- listados y filtros;
- consulta de detalle;
- integración de precio y disponibilidad.

f. Carrito y pedidos

- agregar ítems;
- validar disponibilidad;
- crear pedido;
- manejar estados básicos del pedido.

g. Asistente IA

- responder preguntas del usuario;
- consultar catálogo, inventario y pricing;
- entregar respuestas comerciales contextualizadas.

h. Despliegue técnico

- contenedorización;
- integración de servicios;
- variables de entorno;
- ejecución local o en entorno controlado.

6.2 Alcance excluido

El MVP no incluye:

- pagos reales con pasarela;
- facturación fiscal;
- integración logística real;
- scraping complejo de marketplaces;
- motores avanzados de recomendación basados en ML pesado;
- arquitectura distribuida de alta disponibilidad;
- autenticación empresarial compleja o multitenant;
- analítica avanzada de comportamiento de usuarios.

Estas exclusiones no significan que el proyecto no pueda crecer hacia esos escenarios, sino que no hacen parte de la fase actual y no deben comprometer el tiempo del equipo.

7. Modelo de negocio del sistema

Propósito: Explicar cómo la plataforma genera valor y qué lógica de producto representa.

BookFlow AI Commerce se fundamenta en una idea simple pero poderosa: **el inventario bibliográfico solo se convierte en valor cuando puede ser entendido, comparado, publicado y operado comercialmente**. Por eso, el modelo de negocio del sistema no consiste únicamente en “almacenar libros”, sino en habilitar un ciclo completo de transformación del inventario en activo comercial digital.

7.1 Propuesta de valor central

La plataforma ofrece valor en cuatro dimensiones:

a. Digitalización operativa

Permite pasar de inventarios estáticos o desorganizados a un sistema estructurado y trazable.

b. Mejora de calidad del dato
A través del enriquecimiento, transforma registros pobres en productos más completos y vendibles.

c. Inteligencia comercial
A través del pricing y de la asistencia IA, ayuda a tomar mejores decisiones y a responder mejor al cliente.

d. Conversión operativa
Conecta catálogo, disponibilidad y pedido en un solo flujo coherente.

7.2 Tipo de organizaciones beneficiadas
La solución puede ser valiosa para:

- librerías independientes;
- distribuidores;
- comercializadores de saldos editoriales;
- negocios de libro usado;
- catálogos institucionales;
- emprendimientos de comercio digital enfocados en libros.

7.3 Lógica de monetización potencial
Aunque el proyecto no se construye en esta fase como SaaS productizado, la arquitectura y el enfoque sí permiten imaginar una evolución hacia modelos como:

- licenciamiento por organización;
- suscripción mensual;
- cobro por número de productos o catálogos;
- módulos premium de IA o pricing;
- servicios administrados de publicación y optimización de catálogo.

7.4 Valor de continuidad
El sistema se apoya en la experiencia previa del equipo en el reto de pedidos, pero amplía la complejidad hacia un producto más rico. Esto hace que el proyecto tenga continuidad pedagógica y técnica: no parte desde cero, sino desde una base que ahora se vuelve más robusta, distribuida e inteligente.

7.5 Sentido del producto
En resumen, el modelo de negocio del sistema se puede describir así:

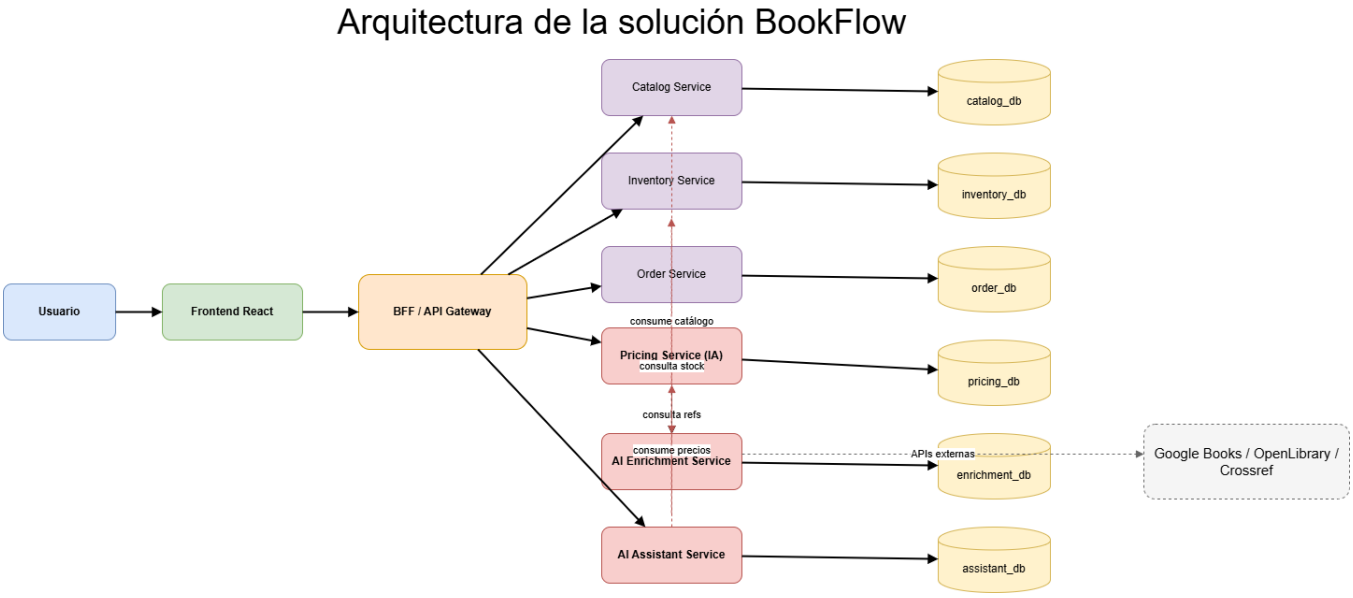
BookFlow AI Commerce convierte inventario bibliográfico en catálogo digital comercializable, con apoyo de IA para mejorar calidad de dato, optimizar precio y asistir al proceso comercial.

No es simplemente un gestor de libros. Es una plataforma que conecta información, decisión y operación.

8. Arquitectura de la solución
Propósito: Definir cómo se organiza el sistema a nivel general, qué componentes existen, cómo se relacionan entre sí y qué decisiones arquitectónicas deben respetarse durante el desarrollo.

La arquitectura de BookFlow AI Commerce se diseña para cumplir simultáneamente cuatro objetivos:
1. permitir construcción incremental por un equipo de 7 a 9 desarrolladores;
2. mantener independencia entre dominios funcionales;
3. facilitar integración de inteligencia artificial sin mezclarla caóticamente con el resto del negocio;
4. asegurar que el sistema pueda ejecutarse, probarse y desplegarse de forma reproducible.

Para cumplir estos objetivos, la solución se implementa bajo una **arquitectura de microservicios independientes con Backend for Frontend (BFF)**, modelo **multi-repo**, persistencia **aislada por servicio** y comunicación **REST síncrona** entre componentes.



Nota: Google Books, Open Library y Crossref aportan metadatos bibliográficos. eBay puede usarse como fuente complementaria de referencia de precios, pero únicamente en escenarios permitidos y sin convertirlo en dependencia indispensable del cálculo.

8.1 Estilo arquitectónico general

La solución no se plantea como un monolito con módulos internos disfrazados de servicios. Cada dominio principal del sistema se implementa como un microservicio independiente, con:

- repositorio propio;
- ciclo de desarrollo propio;
- base de datos o persistencia lógica propia;
- endpoints definidos;
- responsabilidad de negocio específica.

Esto permite que el equipo se distribuya por capacidades sin quedar bloqueado por una sola base de código ni por una única base de datos compartida.

8.2 Componentes principales del sistema

La arquitectura se compone de los siguientes elementos:

8.2.1 Frontend React

Es la capa de presentación del sistema. Se encarga de la experiencia de usuario, tanto en el frente administrativo como en el frente comercial. Sus responsabilidades principales son:

- presentar formularios de carga de inventario;
- permitir navegación del catálogo;
- mostrar detalle de productos;

- exponer paneles administrativos;
- permitir interacción con carrito y pedido;
- ofrecer una interfaz de chat o consulta para el asistente IA.

El frontend no debe contener lógica de negocio crítica. Su función es presentar información, capturar entradas y consumir contratos definidos por el BFF.

8.2.2 BFF / API Gateway

Es el punto único de entrada para el frontend. Su papel no es reemplazar la lógica de negocio de los microservicios, sino:

- simplificar la integración del frontend;
- centralizar contratos;
- agregar respuestas de múltiples servicios;
- adaptar payloads para consumo de interfaz;
- reducir el acoplamiento del frontend con detalles internos del backend.

El BFF se convierte en la capa de orquestación ligera de la solución. No debe contener el corazón del negocio ni asumir responsabilidades de persistencia principal.

8.2.3 Catalog Service

Responsable de exponer y administrar la representación comercial del catálogo. Este servicio debe manejar:

- metadatos de libros y publicaciones;
- categorías o clasificaciones;
- información editorial;
- atributos enriquecidos;
- fichas comerciales consumibles por frontend y por el asistente IA.

Su persistencia se mantiene en catalog_db.

8.2.4 Inventory Service

Responsable del inventario operativo. Este servicio debe gestionar:

- carga de inventario;
- validación de archivos;
- trazabilidad por lote;
- disponibilidad;
- condición física;
- defectos del ejemplar;
- estado del producto desde la perspectiva de existencia y carga.

Su persistencia se mantiene en inventory_db.

8.2.5 Order Service

Responsable del flujo de pedido. Este servicio debe manejar:

- carrito o preparación de orden;
- creación del pedido;
- estados del pedido;
- validaciones básicas del proceso comercial;
- trazabilidad de la orden.

Su persistencia se mantiene en order_db.

8.2.6 Pricing Service

Responsable del cálculo de precios sugeridos. Este servicio no debe ser una simple calculadora hardcodeada. Debe integrar:

- consumo de referencias externas cuando aplique;
- uso de reglas internas del negocio;
- ajuste por condición física del producto;
- trazabilidad del cálculo;
- explicación del resultado.

Su persistencia se mantiene en pricing_db.

8.2.7 AI Enrichment Service

Responsable del enriquecimiento bibliográfico. Su función principal es convertir registros básicos o incompletos en registros más útiles comercialmente, mediante:

- consulta de APIs externas;
- homologación de datos;
- normalización de metadatos;
- selección de atributos relevantes;
- registro de resultados enriquecidos.

Su persistencia se mantiene en enrichment_db o almacenamiento equivalente para trazabilidad.

8.2.8 AI Assistant Service

Responsable de responder preguntas del usuario sobre el catálogo usando datos reales del sistema. Sus funciones incluyen:

- interpretar intenciones de consulta;
- recuperar información desde catálogo, precios e inventario;
- construir respuestas comerciales útiles;
- opcionalmente registrar interacciones para análisis posterior.

Su persistencia se mantiene en assistant_db o almacenamiento equivalente.

8.3 Decisiones de arquitectura

8.3.1 Microservicios independientes

Se adopta este estilo porque el proyecto requiere paralelismo de trabajo y claridad en ownership. Un enfoque monolítico reduciría complejidad inicial, debilitaría el objetivo formativo y limitaría la evolución modular del sistema.

8.3.2 Multi-repo

Cada servicio vive en un repositorio separado. Esto mejora:

- trazabilidad de cambios;
- asignación por desarrollador;
- independencia de despliegue;
- claridad de responsabilidad.

8.3.3 Persistencia por servicio

Cada servicio maneja su propia base o esquema. Esta decisión evita el error de construir microservicios que en realidad dependen de una única base compartida. En desarrollo local, esto puede resolverse con una sola instancia PostgreSQL, pero con bases lógicas separadas.

8.3.4 REST síncrono

La comunicación entre servicios se hará por HTTP REST. Aunque una arquitectura orientada a eventos podría ser interesante, REST ofrece trazabilidad, facilidad de depuración y simplicidad operativa.

8.3.5 IA como servicio

La IA no se mezcla arbitrariamente en cualquier componente. Se encapsula en servicios con responsabilidad clara, lo cual permite aislar integración, pruebas y evolución del comportamiento inteligente.

8.4 Repositorios esperados

La solución puede organizarse bajo repositorios como:

- frontend-bookflow
- bff-bookflow
- catalog-service
- inventory-service
- order-service
- pricing-service
- ai-enrichment-service
- ai-assistant-service

Cada repositorio debe incluir:

- código fuente;
- Dockerfile;
- archivo de dependencias;
- pruebas mínimas;
- README;
- instrucciones de ejecución;
- endpoints o contratos documentados.

8.5 Infraestructura de ejecución

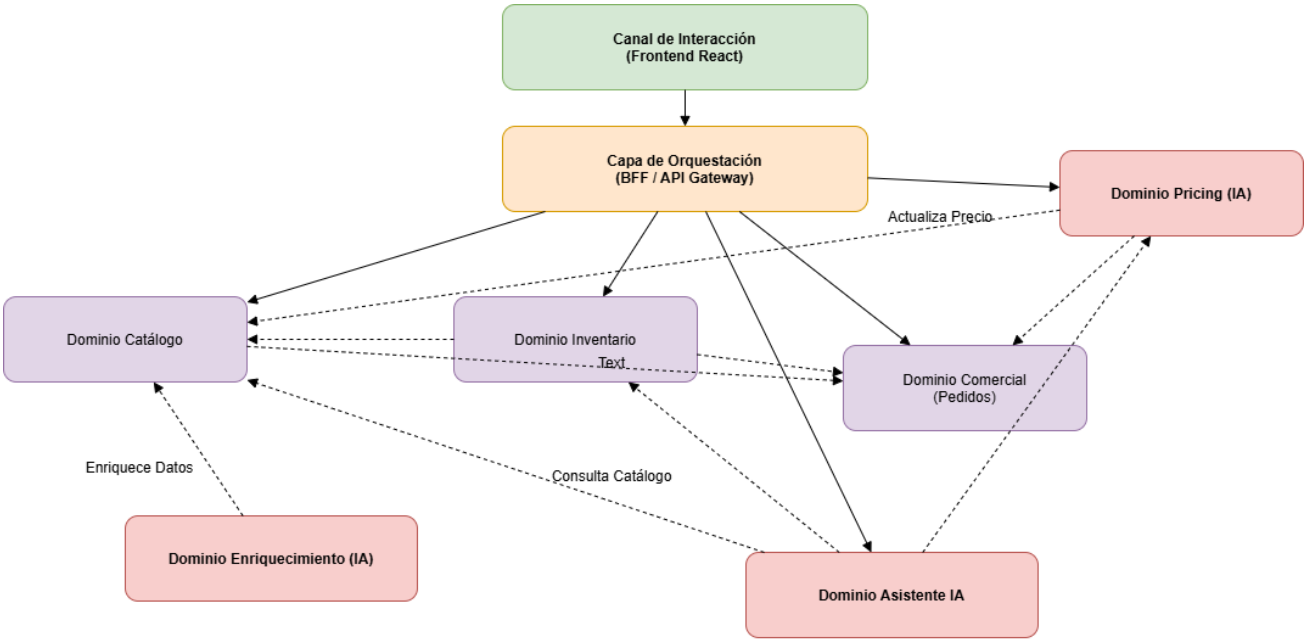
El sistema se integrará en entorno local mediante Docker Compose. Esto permite:

- ejecutar todos los servicios de manera coordinada
- gestionar red interna;
- definir variables de entorno;
- probar integración sin instalación manual fragmentada.

9. Vista lógica de la solución

Propósito: Representar la organización funcional del sistema desde la perspectiva del negocio, identificando los dominios principales, sus responsabilidades y la forma en que interactúan, sin entrar en detalles de despliegue o implementación técnica específica.

La vista lógica de BookFlow AI Commerce permite entender cómo se estructura el sistema en términos de capacidades de negocio, diferenciando claramente los dominios funcionales que componen la solución. A diferencia de la arquitectura de componentes, que describe cómo se construyen los servicios, esta vista se enfoca en qué hace cada parte del sistema y cómo contribuye al flujo de valor.



"La vista lógica organiza el sistema por dominios funcionales y muestra la relación entre capacidades de negocio, sin entrar en detalles de implementación técnica."

9.1 Organización por dominios

El sistema se organiza en los siguientes dominios lógicos:

9.1.1. Canal de Interacción (Frontend)

Este dominio agrupa todos los componentes relacionados con la interacción del usuario final y del administrador con la plataforma. Su responsabilidad es:

- presentar información del catálogo;
- permitir la carga y visualización de inventario;
- facilitar la navegación comercial;
- habilitar la interacción con el asistente IA;
- gestionar la experiencia del usuario en el flujo de pedido.

Este dominio no contiene lógica de negocio crítica, sino que actúa como consumidor de los servicios del backend a través del BFF.

9.1.2. Capa de Orquestación (BFF / API Gateway)

Este dominio actúa como intermediario entre el frontend y los microservicios, y tiene como propósito:

- simplificar el consumo de APIs desde el frontend;
- unificar contratos de acceso;
- agregar información proveniente de múltiples servicios;
- adaptar respuestas a las necesidades de la interfaz.

No debe contener reglas de negocio profundas, pero sí coordinar la interacción entre dominios cuando el frontend lo requiere.

9.1.3. Dominio de Catálogo

Este dominio representa la capa comercial del producto. Es responsable de:

- gestionar la información de libros y publicaciones;
- estructurar fichas de producto;
- exponer datos enriquecidos;
- servir como base de consulta para usuarios y asistentes IA.

El catálogo no depende directamente del inventario para existir, pero se complementa con este para mostrar disponibilidad.

9.1.4. Dominio de Inventario

Este dominio gestiona la existencia física o lógica de los productos. Sus responsabilidades incluyen:

- carga masiva de inventario;
- validación de datos de entrada;
- trazabilidad por lote;
- control de disponibilidad;
- registro de condición física y defectos.

Este dominio es la fuente de verdad sobre qué productos existen y en qué estado.

9.1.5. Dominio Comercial (Pedidos)

Este dominio gestiona el flujo de compra dentro del sistema. Es responsable de:

- construcción del carrito;
- validación de disponibilidad antes de la compra;
- creación de pedidos;
- manejo de estados del pedido;
- trazabilidad del proceso comercial.

Este dominio conecta directamente catálogo, inventario y pricing para ejecutar el flujo de negocio.

9.1.6. Dominio de Pricing (IA)

Este dominio introduce inteligencia en la definición de precios. Sus responsabilidades son:

- calcular precios sugeridos;
- integrar referencias externas;
- aplicar reglas de negocio;
- ajustar precios según condición del producto;
- generar explicaciones del cálculo.

Este dominio es clave para transformar el inventario en un activo comercial competitivo.

9.1.7. Dominio de Enriquecimiento (IA)

Este dominio se encarga de mejorar la calidad de los datos bibliográficos. Sus funciones incluyen:

- consultar fuentes externas;
- normalizar información;
- completar atributos faltantes;
- mejorar la calidad del catálogo;
- alimentar el dominio de catálogo con datos enriquecidos.

Este dominio impacta directamente la calidad de la experiencia del usuario final.

9.1.8. Dominio de Asistencia Inteligente (IA)

Este dominio permite interactuar con el usuario de forma conversacional. Es responsable de:

- interpretar consultas del usuario;
- consultar catálogo, pricing e inventario;
- construir respuestas útiles;
- mejorar la experiencia de navegación y decisión;
- registrar interacciones cuando sea necesario.

Este dominio funciona como una capa transversal que consume múltiples servicios para generar valor.

9.2 Relación entre dominios

Los dominios no operan de forma aislada, sino que se integran a través de flujos de negocio claramente definidos:

- El **inventario** alimenta la existencia de productos que luego pueden ser representados en el **catálogo**.
- El **enriquecimiento IA** mejora los datos del catálogo, aumentando su valor comercial.
- El **pricing IA** toma información del catálogo, inventario y referencias externas para generar precios.
- El **dominio comercial** utiliza catálogo, pricing e inventario para ejecutar pedidos.
- El **asistente IA** consume múltiples dominios para responder consultas del usuario.
- El **BFF** actúa como punto de integración entre frontend y todos los dominios.

Esta organización permite que cada dominio evolucione de forma independiente, manteniendo al mismo tiempo coherencia en el flujo general del sistema.

9.3 Principios de la vista lógica

La definición de esta vista se apoya en los siguientes principios:

- **Separación de responsabilidades:** cada dominio tiene un propósito claro.
- **Bajo acoplamiento:** los dominios interactúan por contratos, no por dependencias internas.
- **Alta cohesión:** cada dominio concentra lógica relacionada.
- **Escalabilidad funcional:** es posible evolucionar dominios sin romper el sistema completo.
- **Soporte a IA como núcleo:** los dominios de IA no son accesorios, sino parte estructural del sistema.

9.4 Valor de la vista lógica dentro del proyecto

La inclusión de esta vista permite:

- alinear al equipo sobre cómo se divide el sistema funcionalmente;
- facilitar la asignación de responsabilidades por dominio;
- mejorar la comprensión del flujo de negocio;
- evitar implementaciones cruzadas o confusas;
- servir como puente entre el entendimiento funcional y la arquitectura técnica.

10. Flujo de Alto Nivel del Sistema

Propósito:
Describir de manera general el recorrido de los datos y las interacciones principales dentro de la plataforma, permitiendo entender cómo los distintos dominios se integran para generar valor sin entrar en el detalle de cada flujo específico.

El flujo de alto nivel de BookFlow AI Commerce representa la secuencia lógica mediante la cual un inventario inicial se transforma en un proceso comercial operable y asistido por inteligencia artificial.

El comportamiento general del sistema se describe a continuación:

1. El administrador realiza la carga de un archivo de inventario a través del sistema.
2. El Inventory Service valida la estructura del archivo, procesa cada registro y genera trazabilidad por lote, incluyendo el registro de errores asociados a filas inválidas.
3. Los registros válidos son almacenados en el sistema y se convierten en insumo para el proceso de enriquecimiento bibliográfico.
4. El AI Enrichment Service consulta fuentes externas, normaliza la información y mejora los metadatos del producto.
5. El Pricing Service analiza la información disponible, consulta referencias cuando aplica y genera un precio sugerido con base en reglas de negocio y condición del producto.
6. El Catalog Service consolida la información del producto, integrando datos enriquecidos y precios sugeridos para su publicación.
7. Los productos son expuestos en el catálogo, donde el usuario puede navegar, filtrar y consultar información relevante.
8. El usuario puede seleccionar productos, agregarlos al carrito y avanzar en el proceso de generación de pedido.
9. El Order Service valida disponibilidad con el Inventory Service y registra la orden dentro del sistema.
10. El AI Assistant Service permite responder consultas del usuario utilizando información real del catálogo, precios y disponibilidad.

Este flujo no representa la implementación técnica detallada, sino una vista integradora que conecta los distintos dominios funcionales del sistema y permite comprender el ciclo completo de operación.

11. Arquitectura de Componentes

Propósito: Documentar con más detalle cómo se estructuran internamente los componentes y cómo se distribuyen sus responsabilidades.

La arquitectura de componentes describe la organización de cada servicio desde el punto de vista del software. Mientras la arquitectura de la solución habla del sistema completo, esta sección baja al nivel de cómo debe construirse cada parte para evitar mezclas incorrectas de responsabilidad.

11.1 Enfoque interno por servicio

Cada microservicio se implementa con una **arquitectura hexagonal ligera**[1], organizada en capas bien definidas. No se documenta como MVC clásico, porque el proyecto no necesita una estructura basada en controlador-modelo-vista dentro del backend, [2]sino una separación clara entre:

- exposición;
- aplicación;
- dominio;
- infraestructura.

11.2 Estructura base recomendada

Cada servicio puede seguir una estructura similar a la siguiente:

```
service-name/
├── app/
│   ├── routers/
```

1 Hexagonal ligera maneja: Exposición (Routers) que recibe la petición | Application (Casos de Uso) Orquesta servicios | llama dominio + infraestructura | Domain (negocio puro) validaciones y cálculos | Infraestructure (todo lo externo): BD, APIs externas, IA, archivos.

2 En backend normalmente sería: Controller → recibe request | Model → datos / base de datos | View → respuesta (a veces JSON).

¿Por qué NO usar MVC aquí?, Porque estás trabajando con microservicios + lógica de negocio más compleja + IA

MVC se queda corto porque: mezcla responsabilidades | mete lógica en lugares incorrectos (controllers gordos) | no separa bien negocio vs infraestructura.

```
│   ├── application/
│   ├── domain/
│   ├── infrastructure/
│   └── main.py
├── tests/
├── Dockerfile
└── requirements.txt
```

11.3 Descripción de capas

11.3.1 Routers[3]

Responsables de:

- recibir requests;
- validar payload básico;
- mapear entradas a casos de uso;
- devolver respuestas HTTP.

No deben contener reglas de negocio complejas ni acceso directo sin control al dominio.

11.3.2 Application

Responsable de:

- orquestar casos de uso;
- coordinar reglas de dominio;
- invocar repositorios o adaptadores;
- mantener lógica de proceso.

Ejemplos:

- crear pedido;
- calcular precio;
- enriquecer un registro;
- listar catálogo.

11.3.3 Domain

Responsable de:

- entidades;
- value objects;
- reglas de negocio;
- validaciones críticas;
- invariantes del dominio.

Aquí vive la lógica que define el comportamiento real del servicio.

11.3.4 Infrastructure

Responsable de:

- acceso a PostgreSQL;
- repositorios concretos;[4]
- clientes HTTP;
- integración con APIs externas;[5]
- proveedores IA;
- lectura de archivos cuando aplique.

---

3 El router es la puerta de entrada del microservicio. Es lo primero que toca una petición http (request), valida el payload (FastApi + Pydantic)

4clases que saben cómo guardar y leer datos de la base de datos

5 Google Books | Open Library | Crossref | eBay (pricing) |

11.4 Arquitectura de componentes por servicio

11.4.1 Frontend React

Responsabilidad

Proveer interfaz de usuario para administración, navegación del catálogo, pedidos y asistente IA.

Componentes internos sugeridos

- páginas de administración;
- páginas de catálogo;
- componentes de tabla;
- componentes de formulario;
- cliente HTTP;
- módulos de estado;
- componentes del chat del asistente.

Regla clave

No implementar lógica de negocio crítica en frontend.

11.4.2 BFF / API Gateway

Responsabilidad

Actuar como fachada del backend para el frontend.

Componentes internos sugeridos

- routers expuestos al frontend;
- servicios de agregación;[6]
- clientes HTTP hacia microservicios;
- DTOs o transformadores de respuesta.

Regla clave

No convertirse en el lugar donde vive toda la lógica del negocio.

11.4.3 Catalog Service

Responsabilidad

Exponer catálogo comercial y metadatos del producto.

Componentes internos

- endpoints de consulta de libros;
- casos de uso de publicación y consulta;
- entidades como Book, Category;
- repositorio sobre catalog_db.

11.4.4 Inventory Service

Responsabilidad

Gestionar stock, carga de inventario y trazabilidad.

Componentes internos

- endpoint de carga masiva;
- validación de archivos;
- manejo de errores de lote;
- entidades como InventoryItem, ImportBatch, ImportError;
- repositorio sobre inventory_db.

11.4.5 Order Service

Responsabilidad

Gestionar el proceso comercial de pedido.

Componentes internos

- endpoints para crear pedido;

---

6 el BFF combina información de varios microservicios en una sola respuesta para el frontend

- casos de uso de transición de estados;
- entidades como Order, OrderItem;
- repositorio sobre order_db.

11.4.6 Pricing Service

Responsabilidad

Definir precios sugeridos explicables.

Componentes internos

- endpoint para cálculo;
- casos de uso de pricing;
- motor de reglas;
- repositorio de decisiones en pricing_db;
- adaptadores para referencias externas.[7]

11.4.7 AI Enrichment Service

Responsabilidad

Enriquecer registros bibliográficos.

Componentes internos

- cliente a APIs externas;
- normalizador;[8]
- mapeador de atributos;
- persistencia de resultados;
- reglas de priorización de fuentes.

11.4.8 AI Assistant Service

Responsabilidad

Responder consultas del usuario usando el sistema como fuente real.

Componentes internos

- endpoint de consulta;
- capa de interpretación de intención;[9]
- conectores a catálogo, pricing e inventario;
- registro de interacciones;
- generación de respuesta final.

10.5 Principios de construcción

- ningún servicio debe acceder directamente a la base de otro;
- toda integración entre dominios debe ocurrir por API;
- cada componente debe tener responsabilidad clara;
- la IA debe trabajar sobre datos reales del sistema;
- toda lógica crítica debe estar en backend, no en frontend.

12. Modelo de Datos

Propósito: Definir las entidades y estructuras mínimas que el sistema requiere para operar de forma coherente.

---

7 componentes que conectan el Pricing Service con fuentes externas de precios

8convertir datos externos en un formato estándar del sistema

9 el componente que entiende qué quiere el usuario cuando hace una pregunta

El modelo de datos debe responder a la arquitectura distribuida. Esto significa que no existe un único esquema global monolítico, sino varios modelos de datos alineados al dominio de cada servicio.

12.1 Modelo de datos del Catalog Service

Entidades principales

- **Book**
  - id
  - title
  - subtitle
  - author
  - publisher
  - publication_year
  - Volumen
  - isbn
  - issn
  - category_id
  - description
  - cover_url
  - enriched_flag
  - published_flag
- **Category**
  - id
  - name
  - description

Propósito

Representar la ficha comercial del producto y permitir consulta, publicación y consumo desde frontend y asistente.

12.2 Modelo de datos del Inventory Service

Entidades principales

- **InventoryItem**
  - id
  - external_code
  - book_reference
  - quantity_available
  - quantity_reserved
  - condition
  - defects
  - observations
  - import_batch_id
- **ImportBatch**
  - id
  - file_name
  - upload_date
  - processed_rows
  - valid_rows
  - invalid_rows
  - status
- **ImportError**
  - id
  - batch_id
  - row_number
  - error_type
  - message

Propósito

Mantener trazabilidad de carga, disponibilidad real y condición física del inventario.

12.3 Modelo de datos del Order Service

Entidades principales

- **Order**
  - id
  - customer_id
  - status
  - created_at
  - updated_at
  - total_amount
- **OrderItem**
  - id
  - order_id
  - book_id
  - quantity
  - unit_price
- **Cart** (si se maneja persistente)
  - id
  - customer_id
  - status
- **CartItem**
  - id
  - cart_id
  - book_id
  - quantity

Propósito

Gestionar la intención de compra y su formalización en pedido.

12.4 Modelo de datos del Pricing Service

Entidades principales

- **PriceReference**
  - id
  - source
  - external_price
  - currency
  - observed_at
  - book_reference
- **PricingDecision**
  - id
  - book_reference
  - suggested_price
  - explanation
  - condition_factor
  - reference_count
  - created_at

Propósito

Mantener trazabilidad del cálculo de precio y permitir explicación posterior.

12.5 Modelo de datos del AI Enrichment Service

Entidades principales

- **EnrichmentRequest**

    - id
    - book_reference
    - requested_at
    - source_used
    - status
- **EnrichmentResult**
    - id
    - request_id
    - normalized_title
    - normalized_author
    - normalized_publisher
    - normalized_description
    - cover_url
    - metadata_json

Propósito

Registrar el proceso de enriquecimiento y sus resultados.

12.6 Modelo de datos del AI Assistant Service

Entidades principales

- **AssistantInteraction**
    - id
    - session_id
    - user_question
    - interpreted_intent
    - answer_text
    - created_at

Propósito

Conservar historial de consultas y permitir análisis posterior.

12.7 Consideraciones de diseño

- los IDs pueden manejarse con UUID;
- la integración entre servicios debe basarse en identificadores estables;
- el modelo debe permitir trazabilidad, no solo operación;
- el diseño debe favorecer explicabilidad, especialmente en pricing e IA.

13. Reglas de Negocio

Propósito: Documentar reglas obligatorias que deben ser respetadas por el sistema, independientemente de la interfaz o implementación concreta.

13.1 Reglas de inventario

1. No se debe considerar disponible un producto si la cantidad disponible es cero.
2. Toda carga de inventario debe estar asociada a un lote.
3. Los errores de carga deben registrarse con trazabilidad de fila.
4. Si un producto es usado, debe registrarse condición física.
5. Si la condición implica defectos, estos deben describirse explícitamente.

13.2 Reglas de catálogo

1. No se debe publicar un producto sin información mínima requerida.
2. Los identificadores bibliográficos deben conservarse si existen.
3. Un producto enriquecido no debe sobrescribir datos críticos sin trazabilidad.

13.3 Reglas de pricing

1. Todo precio sugerido debe dejar explicación asociada.
2. La condición física del ejemplar debe impactar el precio.

3. Si no existen referencias externas válidas, deben aplicarse reglas internas de fallback.
4. El pricing no debe depender de una API externa única como requisito absoluto.

13.4 Reglas de pedidos

1. No se puede confirmar un pedido sin validación de disponibilidad.
2. Las transiciones de estado del pedido deben ser controladas.
3. El pedido debe registrar el precio vigente al momento de la creación.

13.5 Reglas del asistente IA

1. El asistente debe responder usando datos reales del sistema.
2. No debe inventar disponibilidad o precios.
3. Las respuestas deben priorizar exactitud operativa sobre creatividad.
4. Toda consulta relevante debe poder trazarse.

13.6 Reglas arquitectónicas

1. Ningún servicio debe leer directamente la base de otro.
2. El frontend no debe calcular lógica crítica de negocio.
3. El BFF no debe absorber el dominio del sistema.
4. La comunicación entre servicios debe pasar por contratos definidos.
5. APIs externas encapsuladas por adaptadores para evitar acoplamiento fuerte.[10]
6. Pricing con fallback interno: el sistema no debe fallar si una fuente externa no responde.[11]
7. Gestión de condición y defectos como parte del modelo de dominio, no como campo textual residual.[12]
8. Backlog diseñado para independencia por developer dentro del sprint.
9. Documentación distribuida: cada desarrollador documenta su componente en confluence

14. Módulos funcionales del Sistema

Propósito:
Definir los módulos funcionales que componen la solución, identificando las principales capacidades del sistema desde una perspectiva de negocio y su relación con los dominios y microservicios implementados.

Los módulos funcionales de BookFlow AI Commerce representan las capacidades clave necesarias para transformar un inventario bibliográfico en un sistema comercial digital operable, integrando procesos de gestión, inteligencia artificial y flujo comercial.

A continuación, se describen los módulos principales:

14.1 Seguridad y Acceso

Este módulo gestiona la autenticación y control de acceso al sistema. Permite restringir funcionalidades según el tipo de usuario y proteger los endpoints del backend mediante mecanismos como tokens JWT.

14.2 Importación y Control de Calidad del Inventario

Encargado de la carga masiva de inventario mediante archivos estructurados. Incluye validación de formato, procesamiento de registros, identificación de errores por fila y trazabilidad por lote. Garantiza que solo datos válidos ingresen al sistema.

14.3 Gestión de Catálogo Base

Permite la administración de libros y publicaciones, incluyendo metadatos esenciales como título, autor, editorial y clasificación. Este módulo constituye la base del catálogo comercial del sistema.

14.4 Gestión de Condición Física y Defectos

Módulo especializado en registrar el estado físico de los ejemplares, incluyendo deterioros, anotaciones, faltantes u otras condiciones relevantes. Esta información impacta directamente en la percepción del producto y en el pricing.

---

[10] nunca conectes directamente tu lógica a una API externa. MAL (price = requests.get("ebay API")), BIEN (price = pricing_adapter.get_price(isbn)). Esto porque si cambia la API solo cambia el adaptador.

11 el sistema siempre debe poder calcular precio, incluso sin APIs externas

12 la condición del libro NO debe ser texto libre, debe ser estructura del sistema

14.5 Enriquecimiento Bibliográfico (IA)

Responsable de mejorar la calidad de los datos mediante consulta a fuentes externas. Permite completar información faltante como descripciones, portadas, editoriales y categorías, incrementando el valor del catálogo.

14.6 Pricing Inteligente

Implementa la lógica de cálculo de precios sugeridos utilizando referencias externas, reglas de negocio y condición del producto. Incluye trazabilidad y explicación del precio generado, evitando decisiones opacas.

14.7 Catálogo Comercial y Filtros

Permite exponer los productos al usuario final mediante una interfaz navegable, con capacidades de búsqueda, filtrado y visualización detallada de los productos disponibles.

14.8 Carrito de Compras

Facilita la selección de productos por parte del usuario, permitiendo construir una intención de compra previa a la generación del pedido.

14.9 Gestión de Pedidos

Administra la creación y seguimiento de pedidos, incluyendo validación de disponibilidad, registro de la orden y manejo de estados dentro del flujo comercial.

14.10 Asistente Comercial Inteligente (IA)

Permite interactuar con el usuario mediante consultas sobre el catálogo, precios y disponibilidad, utilizando datos reales del sistema para generar respuestas contextualizadas.

14.11 Auditoría y Trazabilidad

Registra eventos relevantes del sistema, como cargas de inventario, decisiones de pricing y generación de pedidos. Permite analizar el comportamiento del sistema y soportar revisiones técnicas y comerciales.

14.12 Configuración y Parámetros del Sistema

Permite definir parámetros operativos del sistema sin necesidad de modificar código, facilitando ajustes en reglas de negocio, comportamiento de servicios o configuraciones generales.

15. Diseño del producto y flujo y Rutas de Interacción

Propósito:
Describir cómo el sistema se organiza desde la perspectiva de uso, diferenciando los principales recorridos funcionales que permiten transformar el inventario en un proceso comercial operable y asistido.

El diseño del producto en BookFlow AI Commerce se estructura en tres rutas principales de interacción, cada una orientada a un tipo de usuario y a un objetivo específico dentro del sistema. Estas rutas no representan la arquitectura técnica, sino la forma en que el sistema es utilizado para generar valor.

15.1 Ruta Administrativa (Operativa)

Esta ruta corresponde a las funcionalidades utilizadas por el administrador o equipo interno, y tiene como objetivo transformar un inventario crudo en un catálogo listo para ser publicado.

El flujo general de esta ruta incluye:

- autenticación en el sistema;
- carga de archivo de inventario;
- validación de estructura y revisión de errores por lote;
- procesamiento y almacenamiento de registros válidos;
- enriquecimiento bibliográfico mediante servicios de IA;
- cálculo de precios sugeridos;
- publicación de productos en el catálogo.

Esta ruta es fundamental para garantizar la calidad del dato y la preparación del producto antes de su exposición comercial.

15.2 Ruta Comercial (Usuario Final)

Esta ruta corresponde a la experiencia del usuario que interactúa con el catálogo con intención de consulta o compra.

El flujo general de esta ruta incluye:

- búsqueda y navegación del catálogo;
- visualización del detalle del producto;
- consulta de disponibilidad y precio;
- selección de productos;
- gestión del carrito;
- generación de pedido dentro del sistema.

Esta ruta materializa el valor comercial del sistema, permitiendo convertir el catálogo en una operación de venta.

15.3 Ruta Asistida (Interacción con IA)

Esta ruta introduce la capa de inteligencia artificial como soporte a la experiencia del usuario.

El flujo general de esta ruta incluye:

- consulta del usuario al asistente;
- interpretación de la intención;
- consulta a servicios internos (catálogo, pricing, inventario);
- generación de respuesta contextualizada;
- posible redirección hacia productos, detalle o carrito.

Esta ruta complementa la navegación tradicional, facilitando la toma de decisiones y mejorando la experiencia del usuario mediante asistencia inteligente.

16. Principios operativos de ejecución

El proyecto se implementa en tres sprints de dos semanas. Cada miembro de la célula presenta en la review un entregable propio, ejecutable y demostrable. La nota del sprint depende de ese entregable individual y de su documentación técnica asociada.

No existe un rol de documentación separado. Cada desarrollador documenta su propio módulo: propósito, endpoints o componentes, decisiones técnicas, supuestos, limitaciones y forma de ejecución.

Tampoco existen roles pasivos. Las tareas transversales como pruebas, soporte de integración o despliegue son responsabilidades complementarias, pero nunca reemplazan la necesidad de entregar una funcionalidad visible.

15. Backlog completo estructurado por sprint

En las siguientes tablas se listan las épicas y las historias de usuario asociadas a cada sprint. Los campos de criterios de aceptación y actividades se dejan intencionalmente vacíos para que la célula los defina como parte del trabajo analítico del sprint. Esta omisión es deliberada y forma parte del ejercicio de diseño y refinamiento.

Sprint 1 — Base arquitectónica y operative

Propósito del sprint: construir la base técnica del sistema, dejar definidos los contratos iniciales entre servicios, habilitar la carga de inventario y el catálogo base, y establecer la estructura que soportará los siguientes sprints.

Sprint	Dev	Microservicio / Módulo	Historia de Usuario	Criterios de Aceptación	Actividades	Entregable
Sprint 1	Dev1	Auth / Seguridad	Como usuario del sistema, quiero autenticarme de forma segura para acceder únicamente a las funcionalidades autorizadas.			Servicio de autenticación funcional y documentado.
Sprint 1	Dev2	Inventory Service	Como administrador, quiero cargar archivos de inventario para registrar productos en lote dentro del sistema.			Microservicio de inventario operativo con carga masiva y trazabilidad por lote.
Sprint 1	Dev3	Catalog Service	Como administrador, quiero registrar y consultar productos bibliográficos para disponer de un catálogo base navegable y estructurado.			Microservicio de catálogo funcional con persistencia propia.
Sprint 1	Dev4	Frontend Administrativo	Como administrador, quiero visualizar el inventario cargado y su estado para poder operar y revisar la información desde una interfaz.			Panel administrativo inicial en React conectado al backend.
Sprint 1	Dev5	Frontend Comercial	Como usuario, quiero visualizar el catálogo base para consultar libros disponibles desde una interfaz simple.			Catálogo web inicial funcional en React.
Sprint 1	Dev6	AI Enrichment Service (Mock)	Como sistema, quiero preparar el servicio de enriquecimiento para desacoplar desde el inicio la lógica de IA del resto del backend.			Servicio de enriquecimiento base listo para integración posterior.
Sprint 1	Dev7	Data Quality / Errores	Como administrador, quiero consultar los errores de carga del inventario para identificar problemas y corregirlos.			Módulo de calidad de datos y errores operando sobre la carga de inventario.
Sprint 1	Dev8	Configuración del Sistema	Como administrador, quiero definir parámetros básicos del sistema para controlar comportamientos generales sin cambiar el código.			Módulo de configuración funcional y reutilizable.
Sprint 1	Dev9	BFF / API Gateway	Como sistema, quiero centralizar el acceso del frontend a los microservicios para simplificar contratos y desacoplar la interfaz del backend interno.			API Gateway / BFF funcional conectado a los primeros microservicios.

Sprint 2 — IA, pricing e integración real

Propósito del sprint: convertir la base técnica en una plataforma inteligente, conectando servicios reales de enriquecimiento, pricing y trazabilidad, y consolidando la integración entre dominios.

Sprint	Dev	Microservicio / Módulo	Historia de Usuario	Criterios de Aceptación	Actividades	Entregable
Sprint 2	Dev1	AI Enrichment Service	Como sistema, quiero enriquecer automáticamente los registros bibliográficos para mejorar la calidad del catálogo sin depender de carga manual completa.			Servicio de enriquecimiento IA real funcionando y documentado.
Sprint 2	Dev2	Limpieza y Normalización de Datos	Como sistema, quiero normalizar y homologar información bibliográfica para evitar inconsistencias y duplicidades en el catálogo enriquecido.			Módulo de normalización y limpieza listo para usarse dentro del flujo de enriquecimiento.
Sprint 2	Dev3	Pricing Service (IA)	Como administrador o sistema, quiero calcular un precio sugerido para cada libro para apoyar decisiones comerciales con lógica explicable.			Motor de pricing funcional, trazable y explicable.
Sprint 2	Dev4	UI de Pricing	Como administrador, quiero visualizar el precio sugerido y eventualmente ajustarlo para poder tomar decisiones comerciales sobre el catálogo.			Panel de pricing funcional conectado al servicio de pricing.
Sprint 2	Dev5	Catálogo Enriquecido	Como usuario, quiero ver productos con información más completa y comercialmente útil para tomar decisiones de compra con mayor confianza.			Catálogo enriquecido visible para el usuario final.
Sprint 2	Dev6	Integración entre Servicios	Como sistema, quiero que los microservicios se comuniquen correctamente por API para soportar un flujo distribuido real.			Integración real y estable entre microservicios principales.
Sprint 2	Dev7	Auditoría de Pricing / IA	Como administrador, quiero entender cómo se llegó a un precio o enriquecimiento para poder confiar en la lógica del sistema.			Módulo de auditoría para decisiones de IA y pricing.
Sprint 2	Dev8	Integración con APIs Externas	Como sistema, quiero conectarme robustamente a fuentes externas para enriquecer y soportar pricing sin colapsar ante fallos.			Integración externa robusta y desacoplada.
Sprint 2	Dev9	Optimización y Persistencia	Como sistema, quiero mantener rendimiento aceptable y persistencia consistente a medida que crecen consultas e integraciones.			Persistencia optimizada para integración y consultas del sprint 2.

Sprint 3 — Flujo comercial completo e IA aplicada al usuario

Propósito del sprint: cerrar el ciclo completo de valor del producto, integrando catálogo, pedido, disponibilidad y asistencia IA en una experiencia funcional de punta a punta.

Sprint	Dev	Microservicio / Módulo	Historia de Usuario	Criterios de Aceptación	Actividades	Entregable
Sprint 3	Dev1	Carrito / Preparación de Orden	Como usuario, quiero agregar productos a un carrito para preparar una intención de compra antes de generar el pedido.			Carrito funcional y usable desde la interfaz.
Sprint 3	Dev2	Order Service	Como usuario, quiero generar un pedido formal para registrar mi intención de compra dentro del sistema.			Order Service completo y funcional.
Sprint 3	Dev3	Flujo End-to-End	Como sistema, quiero ejecutar el recorrido completo desde catálogo hasta pedido para asegurar que el producto sea operable y demostrable.			Flujo end-to-end operable y demostrable.
Sprint 3	Dev4	AI Assistant Service	Como usuario, quiero consultar al sistema por disponibilidad, precio y características de un libro para recibir ayuda comercial contextual.			Asistente IA funcional conectado al sistema real.
Sprint 3	Dev5	Recomendador / Sugerencias	Como sistema, quiero sugerir alternativas relacionadas para enriquecer la experiencia comercial del usuario.			Recomendador básico funcional y demostrable.
Sprint 3	Dev6	BFF Avanzado	Como sistema, quiero centralizar y optimizar la orquestación del frontend para soportar el flujo comercial completo y las respuestas agregadas del producto.			BFF completo y preparado para cierre del proyecto.
Sprint 3	Dev7	Auditoría Global del Sistema	Como administrador, quiero consultar eventos clave del sistema para entender el comportamiento operativo y técnico de la plataforma.			Sistema básico de auditoría global y trazabilidad operativa.
Sprint 3	Dev8	UI Final Integrada	Como usuario, quiero una experiencia completa y coherente para navegar catálogo, interactuar con el asistente, usar carrito y generar pedidos.			Interfaz final integrada y operativa.
Sprint 3	Dev9	Despliegue e Integración Final	Como equipo, quiero ejecutar todo el sistema de forma reproducible para validar la solución integrada y hacer la entrega final.			Sistema completo desplegable y ejecutable de extremo a extremo.

El product Backlog está planificado para 9 developers.

Si finalmente el equipo opera con **7 u 8**, algunas responsabilidades se pueden fusionar así:

- Dev7 + Dev8 en Sprint 2
- Dev5 + Dev7 en Sprint 3
- Dev4 + Dev8 en Sprint 1 si el frente UI es reducido

16. Estrategia de despliegue

La estrategia base contempla despliegue local y de laboratorio mediante Docker Compose. Esto asegura reproducibilidad, reduce errores de ambiente y facilita una demo consistente del producto.

- Ambiente de desarrollo: contenedores para backend, frontend y base de datos.
- Ambiente de demostración: mismo compose con variables ajustadas y datos semilla.
- Persistencia: volumen para PostgreSQL.
- Proxy opcional con Nginx si se desea un punto único de entrada.
- Inicialización controlada mediante scripts de semilla.
- No se contempla despliegue en nube en esta fase

17. APIs requeridas

- Google Books API para volúmenes y metadatos generales.
- Open Library API para consulta pública por ISBN y otros identificadores.
- Crossref REST API para publicaciones seriadas y metadatos académicos.
- eBay Browse API como referencia opcional de mercado para pricing.

El sistema debe funcionar incluso si una de estas APIs falla. Por eso se diseñan adaptadores desacoplados y estrategias de fallback.

18. Seguridad básica del sistema

- Autenticación con JWT.
- Hash de contraseñas y política mínima de credenciales.
- Roles y control de permisos.
- Validación estricta de payloads con Pydantic.
- Control de tamaño y formato en archivos de importación.
- Manejo seguro de variables de entorno.
- CORS controlado y mensajes de error sin fuga de detalles sensibles.
- Auditoría de cambios relevantes.

19. Pruebas recomendadas

- Pruebas unitarias del importador y validaciones de inventario.
- Pruebas unitarias del motor de pricing.
- Pruebas unitarias de reglas de condición y defectos.
- Pruebas de integración de APIs bibliográficas.
- Pruebas de endpoints clave: autenticación, catálogo, carrito, pedido, cotización.
- Pruebas de flujo completo: importación → enriquecimiento → pricing → publicación → carrito → pedido.
- Pruebas del agente sobre disponibilidad, precio y resumen.

20. Métricas de éxito

- Porcentaje de filas importadas correctamente por lote.
- Porcentaje de registros enriquecidos automáticamente.

- Tiempo promedio de publicación de un lote de inventario.
- Porcentaje de títulos con precio sugerido calculado.
- Tiempo de respuesta del catálogo y del agente de consulta.
- Cantidad de cotizaciones y pedidos generados durante pruebas operativas.
- Porcentaje de historias de usuario demostradas en review por sprint.

21. Riesgos técnicos y de negocio con mitigación

Riesgo	Mitigación
API externa inestable	Fallback a reglas internas, cacheo y reintentos controlados.
Datos de inventario sucios o incompletos	Plantilla obligatoria, validaciones por lote y módulo de inconsistencias.
Pricing poco realista por outliers	Filtrado estadístico, límites comerciales y revisión manual.
Bloqueos entre developers	Contratos, mocks y backlog autónomo por sprint.
Módulos invisibles en review	Regla de entregable ejecutable y demostrable por developer.
Catálogo sin transparencia en usados	Campos obligatorios de condición y defectos visibles al cliente.

22. Versión MVP

El MVP debe demostrar el flujo completo mínimo de valor. Incluye autenticación, importación de inventario, gestión de libros/publicaciones, condición y defectos, enriquecimiento básico, pricing sugerido, catálogo navegable, carrito, cotización o pedido, y agente de consulta simple. No requiere pagos reales ni integraciones logísticas.

23. Versión escalable o empresarial futura

En una versión de crecimiento se podría separar el servicio de pricing, incorporar colas de tareas, integrar pasarelas de pago, agregar publicación multi-marketplace, integrar mensajería logística y profundizar analítica comercial. También podría contemplarse un recomendador de títulos y un panel ejecutivo con métricas de rotación y rentabilidad.

24. Recomendaciones de implementación

- Congelar alcance por sprint y no introducir funcionalidades nuevas en la semana final.
- Definir contratos y mocks antes de que cada sprint arranque.
- Exigir demo funcional individual en cada review.
- Obligar a que cada desarrollador documente su módulo y su forma de ejecución.
- Separar claramente datos de semilla, configuración y secretos.
- Probar el flujo completo al menos una vez por sprint.

25. Resultado esperado del proyecto

Al finalizar, la organización debe contar con una plataforma que importe inventario, gestione libros y publicaciones con nivel comercial, refleje condición y defectos, calcule precios sugeridos competitivos, publique catálogo, soporte carrito y flujo comercial, y responda preguntas básicas del cliente mediante un agente conversacional. El proyecto no termina en una presentación; termina en un sistema ejecutable, demostrable y trazable.

26. Glosario técnico

Término	Definición
ISBN	International Standard Book Number; identificador estándar para libros.
ISSN	International Standard Serial Number; identificador estándar para publicaciones seriadas.
Pricing	Conjunto de reglas y datos usados para definir o sugerir precio de venta.
Catálogo	Conjunto de títulos publicados con información comercial visible.
Fallback	Alternativa interna de ejecución cuando falla una dependencia externa.
Outlier	Valor atípico que puede distorsionar una estimación.
API	Interfaz de programación para integración entre sistemas.
JWT	Token firmado usado para autenticación stateless.

27. Referencias bibliográficas y técnicas

1. Google Books API Documentation.
2. Open Library Developer Center and Books/Search APIs.
3. Crossref REST API Documentation.
4. eBay Developers – Browse API Documentation.
5. FastAPI Official Documentation.
6. React Official Documentation.
7. PostgreSQL Official Documentation.
8. Docker Documentation.

28. Cierre

Este documento consolida el planteamiento empresarial de BookFlow AI Commerce y define una base sólida para implementación por sprints, evaluación individual y evolución posterior. Su valor está en combinar enfoque de negocio, claridad técnica y una disciplina operativa que evita dependencias invisibles y entregables ambiguos.